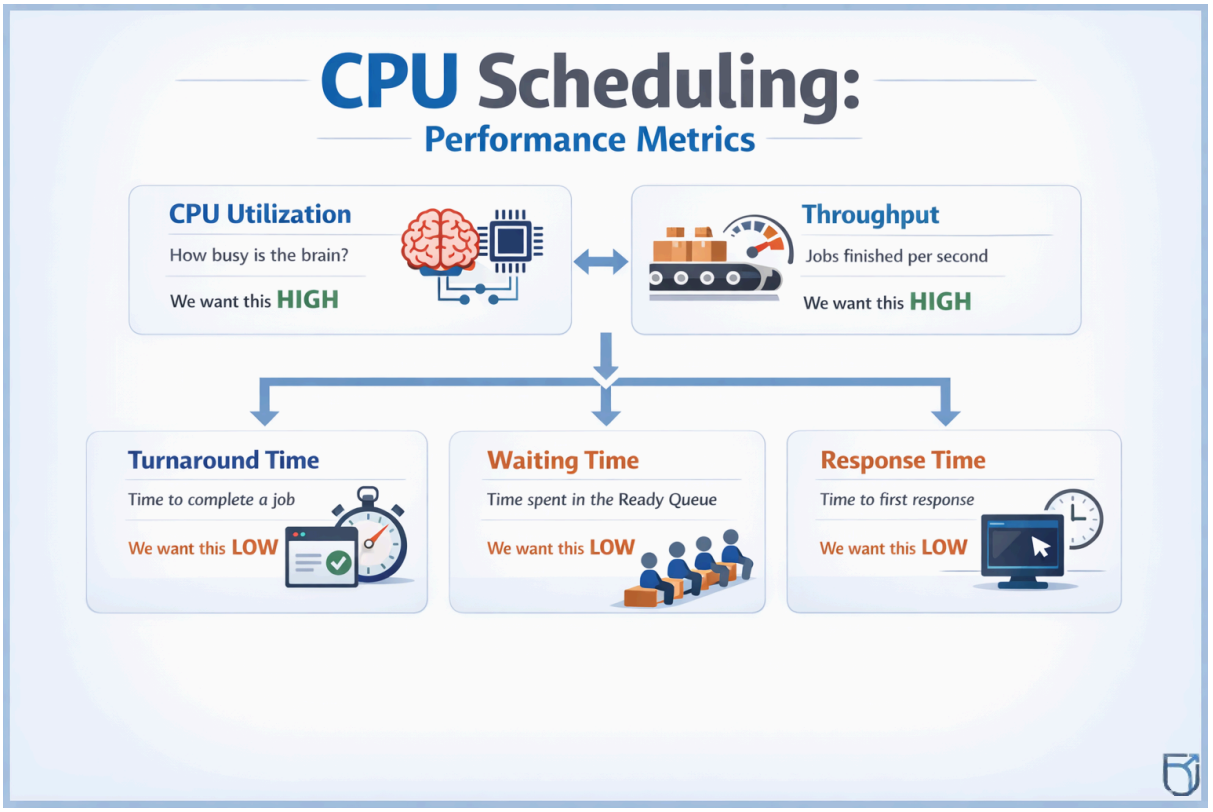
Lecture 07 NOTES – CPU Scheduling Algorithms I: FCFS and SJF

CPU Scheduling

1. The Ready Queue

- **What it is:** CPU scheduling takes place specifically within the **Ready Queue**.
- **The Process:** Multiple processes sit in the main memory (Ready State), waiting for CPU time. The system must select *one* of these processes to transition from **Ready —> Running**.
- **Goal:** To maximize CPU utilization and throughput while minimizing waiting time.



2. Short-Term Scheduler

- **Also known as:** CPU Scheduler.
- **Function:** It is the specific component responsible for selecting which process from the Ready Queue gets the CPU next.
- **Frequency:** It operates very frequently (milliseconds or even faster), which is why it must be extremely fast to avoid wasting CPU time on the scheduling decision itself.

Scheduling Modes: Preemptive vs. Non-Preemptive

A. Non-Preemptive Scheduling

- **Definition:** Once the CPU has been allocated to a process, the process keeps the CPU until it releases it **voluntarily**.
- **When will it be released?**
    1. The process terminates (finishes execution).
    2. The process switches to the waiting state (e.g., waiting for I/O).
- **Pros:** Low context switching overhead.
- **Cons:** Can lead to "starvation" (short jobs wait too long for a long job to finish).
- **Examples:** FCFS (First-Come, First-Served), SJF (Shortest Job First - Non-Preemptive version).

B. Preemptive Scheduling

- **Definition:** The CPU can be taken away ("preempted") from a currently running process **involuntarily** before it finishes, usually to run a higher-priority process.
- **When does it happen?**
    1. A time quantum (time slice) expires (e.g., in Round Robin).
    2. A higher priority process arrives in the Ready Queue.
- **Pros:** Better responsiveness; prevents one process from hogging the CPU.
- **Cons:** Higher overhead due to frequent context switching.
- **Examples:** Round Robin (RR), SRTF (Shortest Remaining Time First), Priority Scheduling (Preemptive mode).

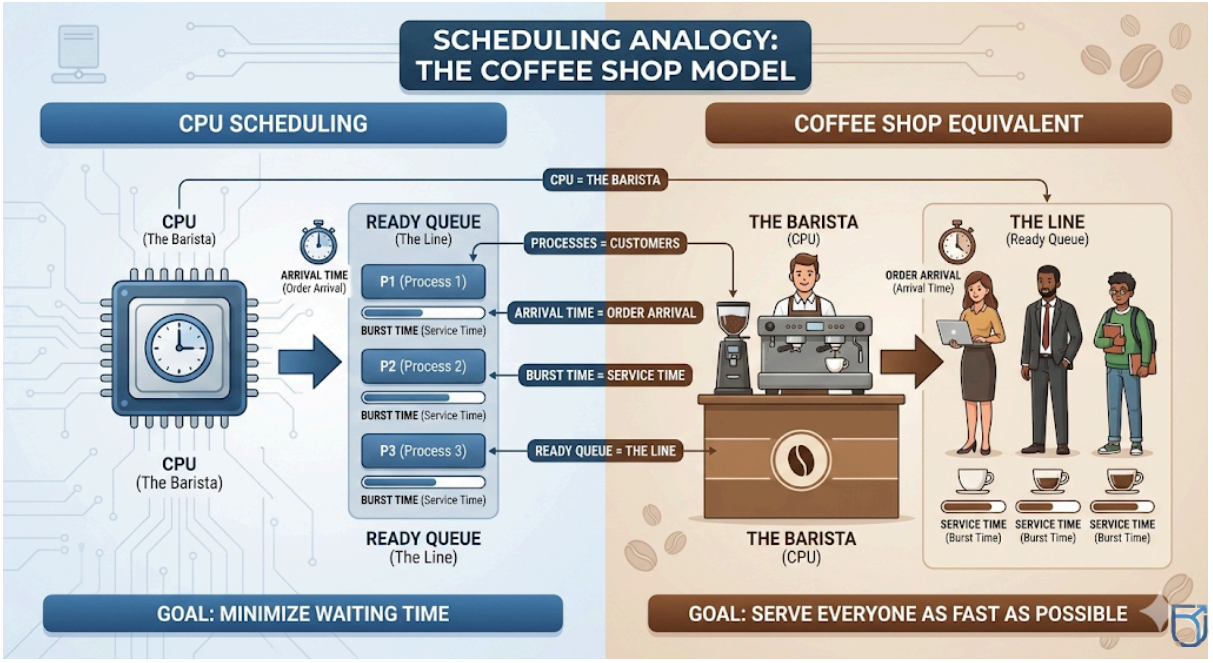
Quick Comparison Table

Feature	**Non-Preemptive**	**Preemptive**
Control	Process holds CPU until done/waiting.	The OS can force a switch.
Context Switching	Low overhead.	High overhead.
Hardware	No special timer required.	Requires a timer interrupt.
Best For	Batch systems.	Time-sharing/Interactive systems.

Scheduling Terms

- **Arrival Time (AT):** The specific time at which a process enters the Ready Queue.
- **Burst Time (BT):** The total time required by the process for execution on the CPU.
- **Completion Time (CT):** The time at which the process finishes its execution.
- **Turnaround Time (TAT):** The total time spent in the system (from arrival to completion).
    - **Formula:**
      TAT = CT - AT
- **Waiting Time (WT):** The total time a process spends waiting in the Ready Queue.
    - **Formula:**
      WT = TAT - BT
- **Average Waiting Time (AWT):** A key metric to measure performance.
    - **Formula:**

$$AWT = \frac{\sum \text{Waiting Time of all processes}}{\text{Total number of processes}}$$



First Come First Served (FCFS)

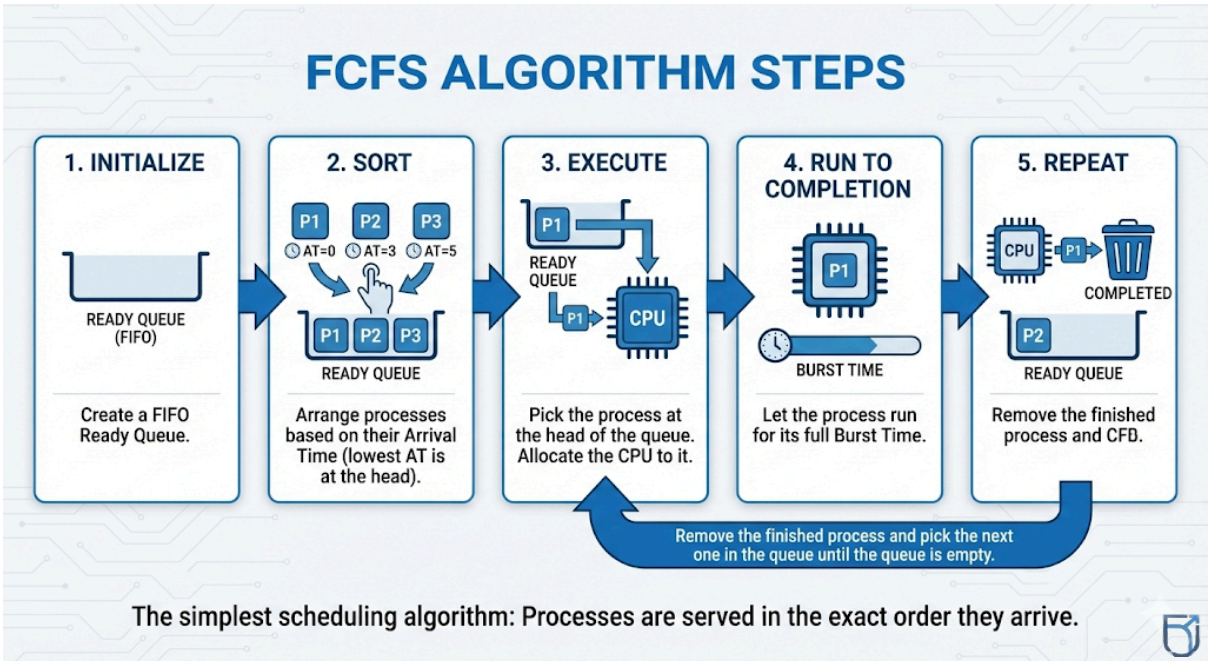
Core Idea

- **Principle:** "First come, first served." The process that arrives first gets the CPU first.
- **Logic:** Execution occurs in the exact order of arrival timestamps.
- **Priority:** There is **no** prioritization based on Burst Time or importance; only Arrival Time matters.

Characteristics

- **Non-Preemptive:** Once the CPU is given to a process, it cannot be interrupted until it finishes its burst.

- **Structure:** Implemented using a **FIFO (First-In, First-Out)** queue.
- **Fairness:** It is simple and fair in the sense that no process is skipped.
- **No Starvation:** Every process will eventually get the CPU (though they might wait a long time if a large process is ahead of them).



FCFS Algorithm Steps

1. **Initialize:** Create a FIFO Ready Queue.
2. **Sort:** Arrange processes based on their **Arrival Time** (lowest AT is at the head).
3. **Execute:**
    - Pick the process at the head of the queue.
    - Allocate the CPU to it.
4. **Run to Completion:** Let the process run for its full **Burst Time**.
5. **Repeat:** Remove the finished process and pick the next one in the queue until the queue is empty.

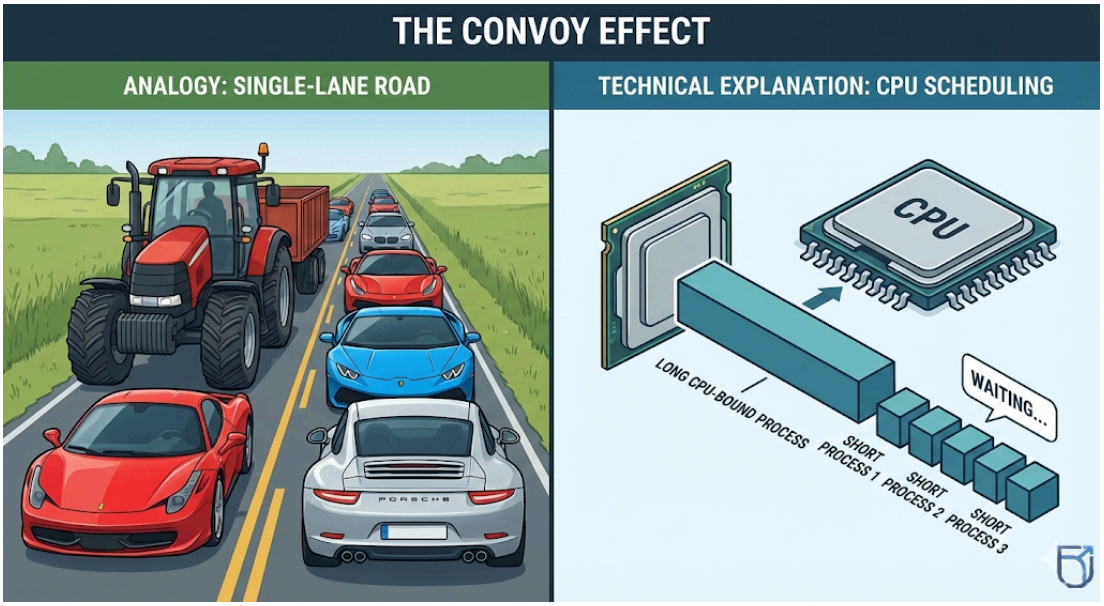
FCFS Performance Analysis

- **Implementation:** It is the easiest algorithm to program and understand (just a simple FIFO queue).
- **Waiting Time:** It generally suffers from a **poor Average Waiting Time (AWT)**, especially if short processes arrive after long ones.
- **Response Time:** Response time is highly unpredictable and often very high.
- **Suitability:** It is **bad for interactive systems** (like desktops or phones) where users expect immediate feedback. It is better suited for batch systems where user interaction is minimal.

FCFS Major Problem: The Convoy Effect

- **Definition:** A phenomenon where several short processes are stuck waiting behind a single long "CPU-bound" process.

- **Analogy:** Imagine a fast Ferrari stuck behind a slow tractor on a single-lane road. The Ferrari is capable of high speed but is limited by the vehicle in front.



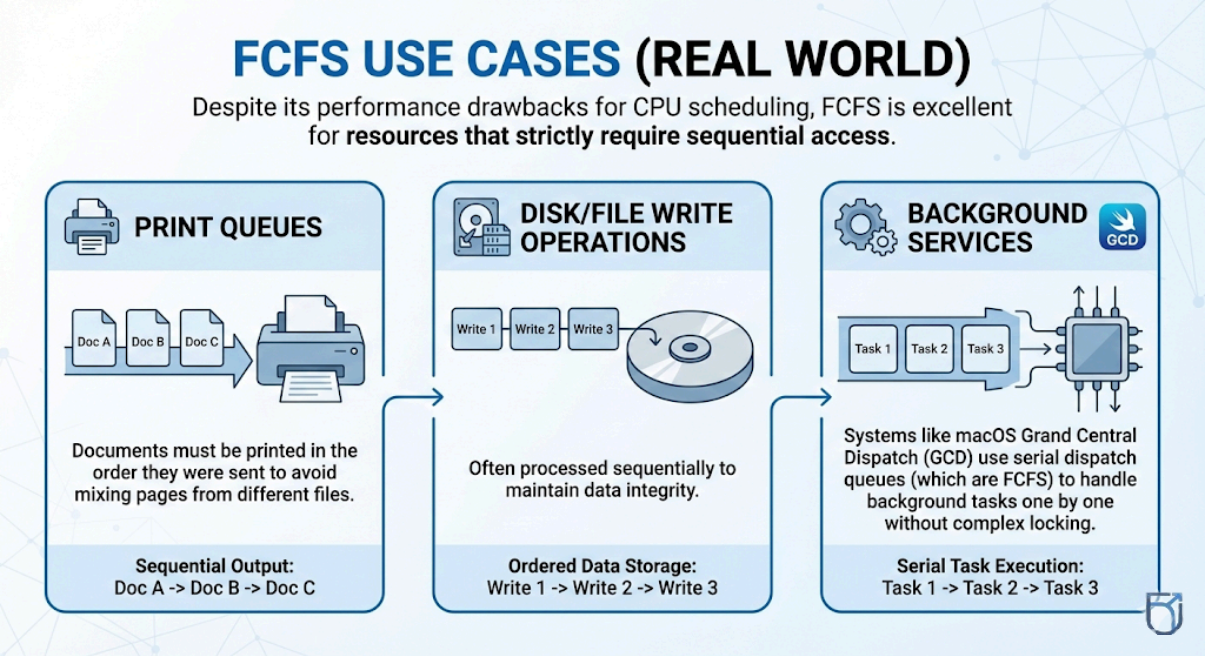
- **Consequences:**
    - **I/O Utilization drops:** While the long process hogs the CPU, I/O devices sit idle.
    - **CPU Utilization drops:** When the long process finally moves to I/O, the CPU sits idle because the short processes finish quickly.
    - **Avg Waiting Time skyrockets:** The wait time for short jobs becomes disproportionately large compared to their execution time.

FCFS Use Cases (Real World)

Despite its performance drawbacks for CPU scheduling, FCFS is excellent for resources that strictly require sequential access:

1. **Print Queues:** Documents must be printed in the order they were sent to avoid mixing pages from different files.
2. **Disk/File Write Operations:** Often processed sequentially to maintain data integrity.

3. **Background Services:** Systems



like **macOS Grand Central Dispatch (GCD)** uses serial dispatch queues (which are FCFS) to handle background tasks one by one without complex locking.

Example 1: First Come First Serve (FCFS) Scheduling

Consider the following set of processes with their Arrival Time (AT) and Burst Time (BT):

Process ID	Arrival Time (ms)	Burst Time (ms)
P0	0	10
P1	1	1
P2	2	1

Using the First Come First Serve (FCFS) CPU scheduling algorithm:

1. Draw the Gantt Chart
2. Calculate the Completion Time (CT) for each process
3. Find the Turnaround Time (TAT) and Waiting Time (WT) for each process
4. Compute the Average Waiting Time (AWT)

Explanation & Solution

Step 1: Scheduling Order (FCFS)

FCFS schedules processes in the order of their arrival time:

Execution Order: P0 → P1 → P2

Step 2: Gantt Chart

P0	P1	P2

0 10 11 12

Step 3: Completion Time (CT)

Process	Completion Time (ms)
P0	10
P1	11
P2	12

Step 4: Turnaround Time (TAT)
Formula:
TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	10	10 − 0 = **10**
P1	1	11	11 − 1 = **10**
P2	2	12	12 − 2 = **10**

Step 5: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	10	10	10 − 10 = **0**
P1	10	1	10 − 1 = **9**
P2	10	1	10 − 1 = **9**

Step 6: Average Waiting Time (AWT)
Formula: AWT = (Sum of Waiting Times) / Number of Processes
AWT=(0+9+9)/3 = 6 ms

Example 2: First Come First Serve (FCFS) Scheduling

Consider the following processes with their Arrival Time (AT) and Burst Time (BT):

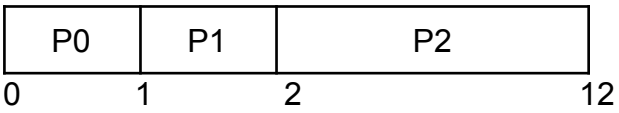
Process ID	Arrival Time (ms)	Burst Time (ms)
P0	0	1
P1	1	1
P2	2	10

Using the First Come First Serve (FCFS) CPU scheduling algorithm:

1. Draw the Gantt Chart
2. Calculate the Completion Time (CT) for each process
3. Compute the Turnaround Time (TAT) and Waiting Time (WT)
4. Find the Average Waiting Time (AWT)

Explanation & Solution
Step 1: Scheduling Order (FCFS)
In FCFS, processes are executed in the order in which they arrive.
Order of execution:
P0 → P1 → P2

Step 2: Gantt Chart



Step 3: Completion Time (CT)

Process	Completion Time (ms)
P0	1
P1	2
P2	12

Step 4: Turnaround Time (TAT)
Formula:
TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	1	1 − 0 = **1**
P1	1	2	2 − 1 = **1**
P2	2	12	12 − 2 = **10**

Step 5: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	1	1	1 − 1 = **0**
P1	1	1	1 − 1 = **0**
P2	10	10	10 − 10 = **0**

Step 6: Average Waiting Time (AWT)
Formula: AWT= Sum of Waiting Times/Number of Processes
AWT=(0+0+0)/3 = 0 ms

Example 3 (What If Case): First Come First Serve (FCFS) Scheduling

Consider the following processes with Arrival Time (AT) and Burst Time (BT):

Process ID	Arrival Time (ms)	Burst Time (ms)
P0	0	10
P1	0	1
P2	0	1

All processes arrive at the same time (AT = 0 ms).
Assume the order of processes in the ready queue is P0 → P1 → P2.

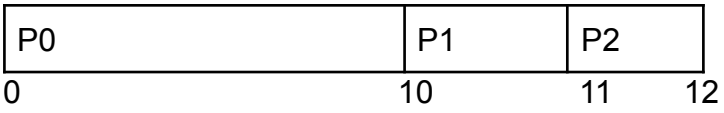
Using the First Come First Serve (FCFS) scheduling algorithm:

1. Draw the Gantt Chart
2. Calculate the Completion Time (CT)
3. Find the Turnaround Time (TAT) and Waiting Time (WT)
4. Compute the Average Waiting Time (AWT)

Explanation & Solution
Step 1: Scheduling Order (FCFS)
When all processes arrive at the same time, FCFS schedules them in the order they appear in the ready queue.

Execution Order: P0 → P1 → P2

Step 2: Gantt Chart



Step 3: Completion Time (CT)

Process	Completion Time (ms)	Process
P0	10	P0
P1	11	P1
P2	12	P2

Step 4: Turnaround Time (TAT)
Formula:
TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	10	10 − 0 = **10**
P1	0	11	11 − 0 = **11**

P2	0	12	12 − 0 = **12**

Step 5: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	10	10	10 − 10 = **0**
P1	11	1	11 − 1 = **10**
P2	12	1	12 − 1 = **11**

Step 6: Average Waiting Time (AWT)
Formula: Sum of Waiting Times/Number of Processes
AWT=(0+10+11)/3 = 7 ms

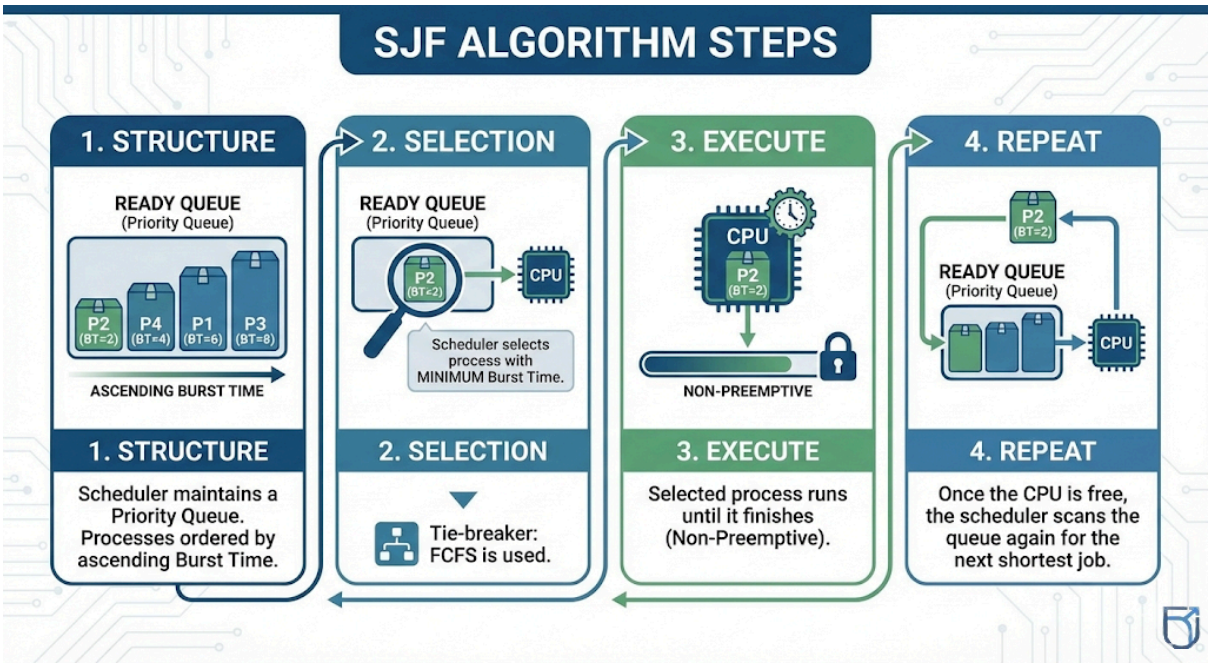
Shortest Job First (SJF)

Core Idea

- **Principle:** "Greedy" approach. The process with the **smallest CPU Burst Time (BT)** is selected next.
- **Logic:** Short jobs are allowed to "jump the line" ahead of long jobs.
- **Requirement:** The operating system must know (or estimate) the length of the next CPU burst for every process in the queue.

Characteristics

- **Non-Preemptive:** In standard SJF, once a process starts, it runs until the burst is complete (even if a shorter job arrives during execution).
- **Optimality:** SJF is **probably optimal** for minimizing Average Waiting Time. It gives the lowest possible AWT for a given set of processes.
- **Criterion:** Selection is based purely on **Burst Time**, ignoring Arrival Time (once the process is in the queue).



SJF Algorithm Steps

- **Structure:** Instead of a simple FIFO queue, the scheduler maintains a **Priority Queue** where processes are ordered by ascending Burst Time.
- **Selection:**
    1. The scheduler looks at all available processes in the Ready Queue.
    2. It selects the process with the **minimum Burst Time**.
    3. If there is a tie in Burst Times, FCFS is used to break the tie.
- **Execution:** The selected process runs until it finishes (Non-Preemptive).
- **Repeat:** Once the CPU is free, the scheduler scans the queue again for the next shortest job.

SJF Advantages

- **Optimal Average Waiting Time:** Mathematically proven to yield the minimum average waiting time for a given set of processes.
- **Responsiveness:** Short jobs (which are often interactive tasks) get processed quickly, improving system responsiveness compared to FCFS.
- **Throughput:** By clearing short jobs quickly, it increases the number of processes completed per unit of time (high throughput).

SJF Problems

- **The Prediction Problem:** The OS cannot know the exact Burst Time of a process *before* it runs. It is impossible to implement perfectly in general-purpose scheduling.
- **Starvation (Indefinite Blocking):** If short processes keep arriving, a long process may never get the CPU. It will stay at the back of the priority queue forever.
- **Fairness:** It is inherently unfair to long jobs (CPU-bound processes).

SJF Convoy Effect Comparison

- **Convoy Effect Reduced:** SJF largely solves the FCFS "Convoy Effect" because short jobs don't get stuck behind long ones; they jump ahead.
- **Trade-off:** While the "convoy" (traffic jam) is cleared, the "heavy trucks" (long processes) might be pulled over to the side of the road indefinitely (Starvation).

Example 1: Shortest Job First (SJF) Scheduling

Consider the following set of processes with their Arrival Time (AT) and Burst Time (BT):

Process ID	Arrival Time (ms)	Burst Time (ms)
P0	0	10
P1	1	1
P2	2	1

Using the Shortest Job First (SJF) scheduling algorithm (non-preemptive):

1. Draw the Gantt Chart
2. Calculate the Completion Time (CT)
3. Find the Turnaround Time (TAT) and Waiting Time (WT)
4. Compute the Average Waiting Time (AWT)

Explanation & Solution

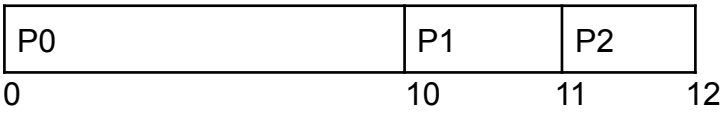
Step 1: Scheduling Decision (SJF – Non-Preemptive)

At time 0 ms, only P0 has arrived, so it must start execution. SJF does not preempt a running process. Even though P1 and P2 have shorter burst times, they arrive after P0 has started.

Execution Order: P0 → P1 → P2

Hence, SJF behaves like FCFS in this case.

Step 2: Gantt Chart



Step 3: Completion Time (CT)

Process	Completion Time (ms)
P0	10
P1	11
P2	12

Step 4: Turnaround Time (TAT)

Formula:

TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	10	10 − 0 = **10**
P1	1	11	11 − 1 = **10**
P2	2	12	12 − 2 = **10**

Step 5: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	10	10	10 − 10 = **0**
P1	10	1	10 − 1 = **9**
P2	10	1	10 − 1 = **9**

Step 6: Average Waiting Time (AWT)
AWT = (0+9+9)/3 = 6 ms

Example 2 (What If Case): Shortest Job First (SJF) Scheduling

Consider the following processes with their Arrival Time (AT) and Burst Time (BT):

Process ID	Arrival Time (ms)	Burst Time (ms)
P0	0	10
P1	0	1
P2	0	1

All processes arrive at time 0 ms.

Using the Shortest Job First (SJF) scheduling algorithm (non-preemptive):

1. Draw the Gantt Chart
2. Calculate the Completion Time (CT)
3. Find the Turnaround Time (TAT) and Waiting Time (WT)
4. Compute the Average Waiting Time (AWT)

Explanation & Solution

Step 1: Scheduling Decision (SJF)

At time 0 ms, all processes are available in the ready queue. SJF selects the process with the shortest burst time first.

Burst times:

P1 = 1 ms

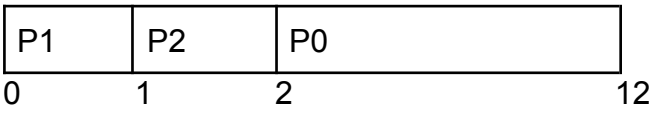
P2 = 1 ms

P0 = 10 ms

Since P1 and P2 have the same shortest burst time, they are executed based on FCFS order among equals.

Execution Order: P1 → P2 → P0

Step 2: Gantt Chart



Step 3: Completion Time (CT)

Process	Completion Time (ms)	Process	Completion Time (ms)
P1	1	P1	1
P2	2	P2	2
P0	12	P0	12

Step 4: Turnaround Time (TAT)

Formula:

TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	12	12 − 0 = **12**
P1	0	1	1 − 0 = **1**
P2	0	2	2 − 0 = **2**

Step 5: Waiting Time (WT)

Formula:

WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	12	10	12 − 10 = **2**
P1	1	1	1 − 1 = **0**
P2	2	1	2 − 1 = **1**

Step 6: Average Waiting Time (AWT)

AWT = (2+0+1)/3 = 1 ms

Example (What If Case): Shortest Remaining Time First (SRTF) Scheduling
Consider the following processes with their Arrival Time (AT) and Burst Time (BT):

Process ID	Arrival Time (ms)	Burst Time (ms)
P0	0	10
P1	0	1
P2	0	1

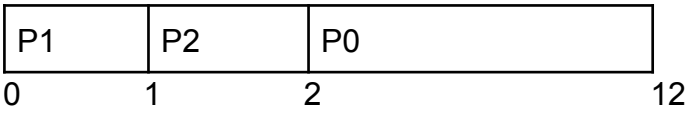
All processes arrive at time 0 ms.
Using the Shortest Remaining Time First (SRTF) scheduling algorithm:

1. Draw the Gantt Chart
2. Calculate the Completion Time (CT)
3. Find the Turnaround Time (TAT) and Waiting Time (WT)
4. Compute the Average Waiting Time (AWT)

Explanation & Solution
Step 1: Scheduling Decision (SRTF)
SRTF is the preemptive version of SJF. At time 0 ms, all processes are available:
P1 = 1 ms
P2 = 1 ms
P0 = 10 ms
The process with the shortest remaining time is selected first.
Between P1 and P2 (equal burst times), FCFS order is followed.

Execution Order: P1 → P2 → P0
Since no new process arrives later with a shorter remaining time, no preemption occurs.

Step 2: Gantt Chart



Step 3: Completion Time (CT)

Process	Completion Time (ms)	Process	Completion Time (ms)
P1	1	P1	1
P2	2	P2	2

P0	12	P0	12

Step 4: Turnaround Time (TAT)
Formula:
TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	12	12 − 0 = **12**
P1	0	1	1 − 0 = **1**
P2	0	2	2 − 0 = **2**

Step 5: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	12	10	12 − 10 = **2**
P1	1	1	1 − 1 = **0**
P2	2	1	2 − 1 = **1**

Step 6: Average Waiting Time (AWT)
AWT = (2+0+1)/3 = 1 ms

Preemptive SJF: SRTF

- **Name:** Shortest Remaining Time First.
- **Logic:** It is the **preemptive** version of SJF.
- **The Check:** When a new process arrives in the Ready Queue, the scheduler compares the **remaining time** of the currently running process vs. the **burst time** of the new arrival.
- **Action:** If the new process is shorter than the *remaining* time of the current one, the CPU is preempted (taken away) and given to the new process.

SRTF Characteristics

- **Most Optimal:** It provides the absolute minimum Average Waiting Time among all scheduling algorithms.
- **Overhead:** It requires frequent context switching and constant comparison of remaining times, leading to higher system overhead than non-preemptive SJF.
- **Risk:** Like SJF, it suffers from severe starvation potential for long processes.

FCFS vs SJF vs SRTF (Conceptual)

Algorithm	Philosophy	Key Strength	Key Weakness
FCFS	"First come, first served"	Simple, Fair (no starvation).	Convoy Effect, High Wait Time.
SJF	"Shortest job first"	Low Avg Wait Time.	Future knowledge required, Starvation.
SRTF	"Shortest remaining time"	**Lowest** Avg Wait Time.	High Overhead, Complex.

Burst Time Prediction Issue

- **The Dilemma:** Since we can't know the future, we must **estimate** the next CPU burst.
- **Method:** We assume the next burst will be similar to the previous bursts.
- Exponential Averaging: The standard formula used to predict the next burst

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Real OS Perspective (macOS & Others)

- **Reality Check:** Modern operating systems (Windows, Linux, macOS) do **not** use pure SJF or FCFS because of the prediction and starvation issues.
- **The Solution: Multilevel Feedback Queue (MLFQ).**
    - It mimics SJF behavior without knowing burst times.
    - Processes start with high priority (short time slice). If they use the whole slice (acting like a long job), they are demoted to a lower priority queue.
- **macOS Specifics:**
    - macOS uses **Quality of Service (QoS)** classes.
    - **User Interactive** (Animation/UI) —-> High Priority (Short bursts).
    - **Background/Utility** (Backups/Indexing) —-> Low Priority (Long bursts, can wait).